



KIẾN TRÚC VÀ THIẾT KẾ PHẦN MỀM

Chương 9: Một số lưu ý trong kiến trúc phần mềm

ThS. Nguyễn Văn Hữu Hoàng

I NỘI DUNG

- 9.1 Dependency Inversion là gì?
- 9.2 Inversion of Control (IoC)
- 9.3 Dependency Injection (DI)
- 9.4 IoC Container

| 9.1 Dependency Inversion là gì?

Quy tắc SOLID trong OOP Design

- Single responsibility principle:
 - Có ý nghĩa là một class chỉ nên giữ một trách nhiệm duy nhất.
- Open/closed principle:
 - Có thể thoải mái mở rộng 1 class nhưng không được sửa đổi bên trong class đó.
- Liskov substitution principle:
 - Bất cứ instance nào của class cha cũng có thể được thay thế bởi instance của class con của nó mà không làm thay đổi tính đúng đắn của chương trình.
- Interface segregation principle:
 - Thay vì dùng 1 interface lớn, ta nên tách thành nhiều interface nhỏ, với nhiều mục đích cụ thể Client không nên phụ thuộc vào interface mà nó không sử dụng.
- Dependency inversion principle:
 - Các module, class cấp cao không nên phụ thuộc vào module, class cấp thấp hơn mà nên phụ thuộc (giao tiếp) với nhau thông qua một abstraction (Interface).
 - Abstraction không nên phụ thuộc vào chi tiết. Chi tiết nên phụ thuộc vào abstraction.

I Dependency Inversion

- Một ví dụ để dễ hiểu về Dependency Inversion: Mặc dù có hình dáng khác nhau nhưng đèn huỳnh quang hoặc đèn tròn đều có thể dễ dàng thay thế nhau. Ở đây ta có thể hiểu interface chính là đuôi tròn còn implementation là bóng đèn tròn và bóng đèn huỳnh quang.
- Ta có thể hoán đổi giữa 2 loại bóng đèn bởi ổ điện chỉ quan tâm tới interface là cái đuôi tròn chứ không cần quan tâm tới implementation. Trong code cũng tương tự như vậy, khi áp dụng Dependency Inversion, bạn chỉ cần quan tâm tới interface

I Dependency Inversion

- Giả sử có:
 - Một class `Circle` (đây là một class low-level) có public cho tôi 2 method: `getPerimeter()` và `getArea()` để tính chu vi và diện tích của hình tròn.
 - Một class `ShapeManager` (đây là một class high-level) sẽ public 2 method: `calculatePerimeter()` và `calculateArea()`;
- Muốn in ra màn hình diện tích của hình tròn có bán kính cố định nào đó, implement như sau:

```
public class ShapeManager {  
    private Circle circle;  
    public ShapeManager() {  
    }  
    public void setCircle(Circle circle) {  
        this.circle = circle;  
    }  
    public float calculatePerimeter() {  
        return circle.getPerimeter();  
    }  
    public float calculateArea() {  
        return circle.getArea();  
    }  
}  
  
public static void main(String[] args) {  
    ShapeManager manager = new ShapeManager();  
    manager.setCircle(new Circle(5));  
    System.out.println("Circle with perimeter and area: "  
        + manager.calculatePerimeter() + ":" +  
        manager.calculateArea());  
}
```

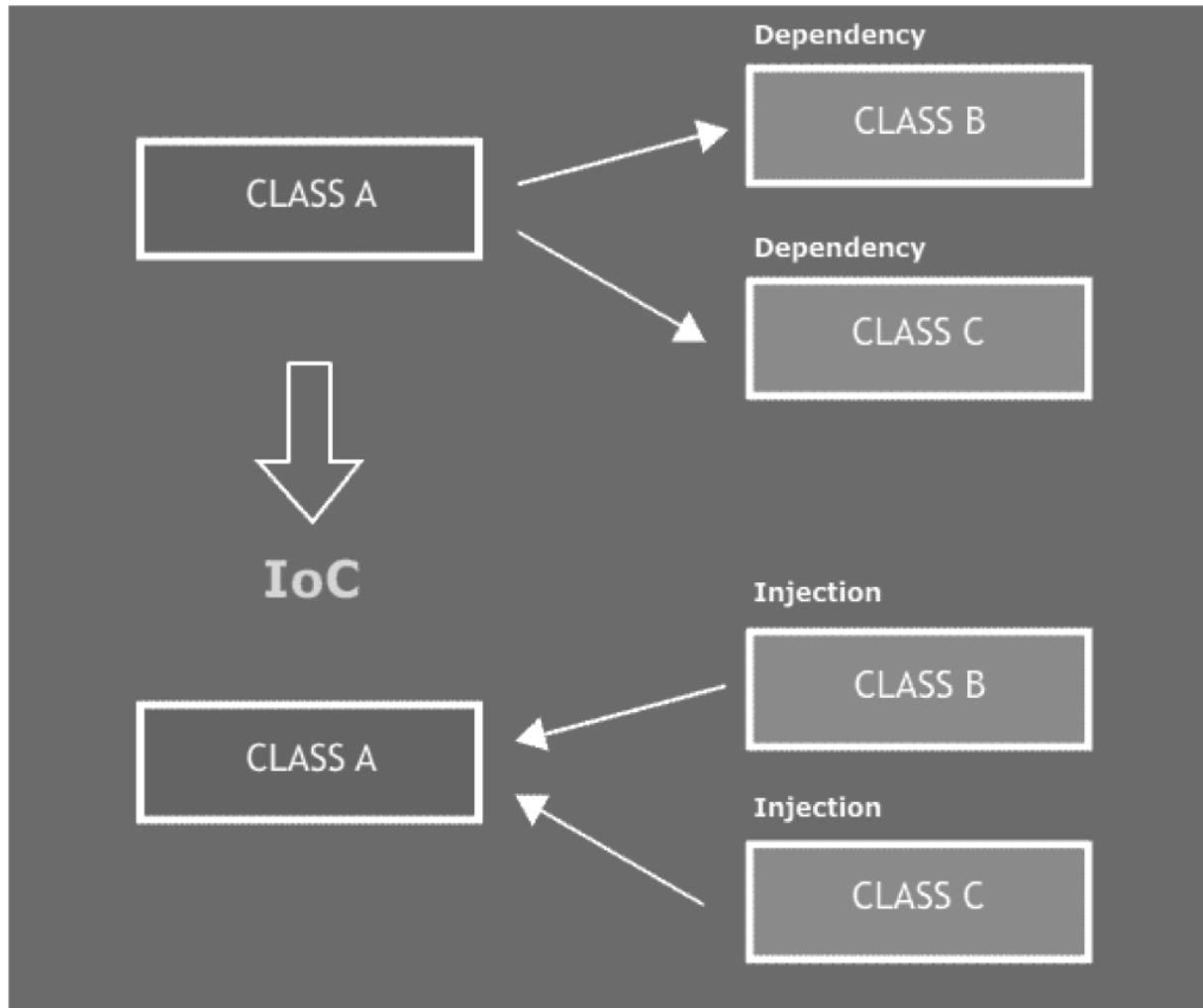
I Dependency Inversion

- Code thoát nhìn không có vấn đề gì cả và vẫn cho ra kết quả đúng. Thế nhưng, nếu tôi muốn mở rộng class **ShapeManager** có thể tính được nhiều hình khác nhau: hình vuông, hình chữ nhật, hình thoi ... thì phải tạo thêm nhiều đối tượng trong class **ShapeManager**.
- Càng lúc, class này sẽ phình to ra tới mức không thể hình dung ra được. Rõ ràng, chúng ta đang thấy class high-level ở đây là **ShapeManager** đang phụ thuộc vào class low-level.
- **Dependency Inversion** muốn các module này không nên phụ thuộc vào nhau và giao tiếp với nhau thông qua một abstraction (Interface). Bởi lẽ đó, **Inversion of Control** ra đời để làm nhiệm vụ đó.

I 9.2 Inversion of Control

- **Inversion of Control (IoC)** là một nguyên lý thiết kế trong công nghệ phần mềm trong đó các thành phần nó dựa vào để làm việc bị đảo ngược quyền điều khiển khi so sánh với lập trình hướng thủ tục truyền thống.
- IoC là một design pattern để hiện thực hóa hay implement nguyên lý thiết kế Dependency Inversion nêu trên. Dĩ nhiên, nó sẽ tuân thủ đầy đủ những nguyên tắc của Dependency Inversion phát biểu.
- IoC sẽ **không quan tâm đến việc Service được khởi tạo như thế nào** mà **chỉ quan tâm đến những gì mà nó cung cấp** thông qua một abstraction. Điều này tuân thủ chặt chẽ nguyên lý của Dependency Inversion, tức là các module high-level chỉ phụ thuộc hay giao tiếp với các module low-level thông qua một abstraction, và chính module high-level sẽ không cần biết module low-level sẽ được khởi tạo như thế nào mà chỉ cần biết những gì nó cung cấp.

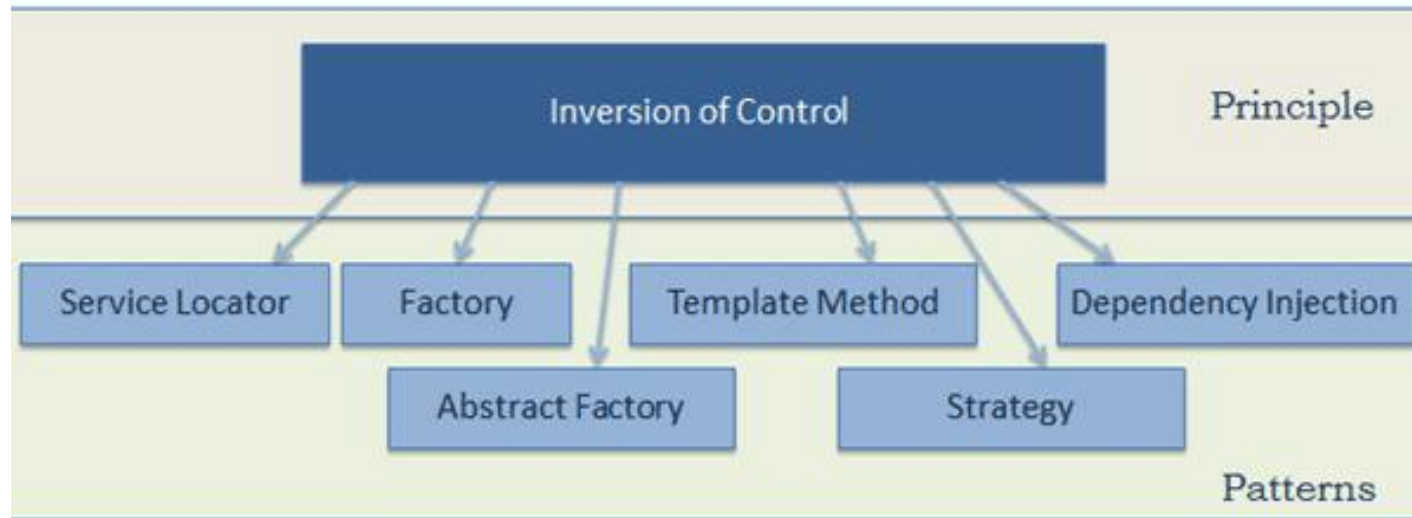
| Inversion of Control



- Hình bên là **Class A** ở hai trường hợp, áp dụng IoC và không. Lớp này nó làm việc phụ thuộc vào hai lớp **Class B** và **Class C** (B và C gọi là các dependency).
- Ở mô hình không IoC, Class A khi cần **chủ động tạo ra đối tượng** lớp Class B và Class C (nó nắm quyền khởi tạo, điều khiển).
- Với mô hình IoC thì class A **không tự khởi tạo cũng không chịu trách nhiệm** quản lý Class B, Class C. Nó nhận được hai dependency này từ bên ngoài thông qua một cơ chế nào đó (như bằng setter, bằng tham số hàm tạo lớp A, bằng gán thuộc tính ...)

I Triển khai nguyên lý IoC

- Triển khai nguyên lý IoC thường thực hiện bởi các Framework theo từng loại ngôn ngữ lập trình, các mô hình lập trình (pattern) triển khai từ IoC như:
 - Service Locator
 - Factory
 - Dependency Injection: Constructor DI, Property DI, Method DI.
 - Events
 - ...



I Một số đặc điểm nổi bật khi ứng dụng IOC:

- Tăng sự linh hoạt và khả năng bảo trì của code: sử dụng IOC làm giảm đi sự phụ thuộc giữa những đối tượng từ đó tăng khả năng tái sử dụng và bảo trì code dễ hơn.
- Giảm đi sự ràng buộc giữa những thành phần trong code: giúp quá trình mở rộng cũng như thay đổi các chức năng của ứng dụng sẽ trở nên dễ dàng hơn.
- Dễ dàng kiểm tra và debug: Nhờ vào khả năng tự động hóa của IOC, việc kiểm thử và debug luôn dễ dàng và đơn giản hơn.
- Giảm thiểu lỗi do sai sót: áp dụng **IOC container**, tất cả đối tượng sẽ được quản lý tự động và đồng nhất. Điều này giúp giảm thiểu tối đa sai sót do con người gây ra.
 - IOC container có nhiệm vụ bổ sung những đối tượng vào trong controller hoặc service.
 - Đồng thời giúp cho quá trình quản lý các đối tượng sẽ trở nên dễ dàng hơn.

I Ví dụ IoC

- Trở lại với ví dụ trước, trong trường hợp muốn mở rộng class `ShapeManager` hỗ trợ thêm các hình khác như hình vuông, hình chữ nhật -> tiến hành implement nó theo IoC.
- Tạo một `interface Shape`. Interface này sẽ làm nhiệm vụ kết nối `ShapeManager` (high-level module) với các implementation của `Shape` (low-level module) hay nói cách khác `ShapeManager` chỉ phụ thuộc vào `interface Shape` mà ko cần quan tâm nó được khởi tạo ở đâu và bằng cách nào.

```
public interface Shape {  
    float getPerimeter();  
    float getArea();  
}
```

I Ví dụ IoC

- Tạo các implementation của Shape. Đây chính là các Service (low-level module) cần dùng trong high-level module.

```
public class Circle implements Shape {  
    private static final float PI = 3.1415f;  
    private int radius;  
    public Circle(int radius) {  
        this.radius = radius;  
    }  
    @Override  
    public float getPerimeter() {  
        return radius * 2 * PI;  
    }  
    @Override  
    public float getArea() {  
        return (float)(Math.pow(radius, 2)*PI);  
    }  
}
```

```
public class Square implements Shape {  
    private int size;  
    public Square(int size) {  
        this.size = size;  
    }  
    @Override  
    public float getPerimeter() {  
        return size * 4;  
    }  
    @Override  
    public float getArea() {  
        return (float) Math.pow(size, 2);  
    }  
}
```

I Ví dụ IoC (tt)

Modify class ShapeManager.

```
public class ShapeManager {
    private Shape shape;
    public void setShape(Shape shape) {
        this.shape = shape;
    }
    public float calculatePerimeter() {
        return this.shape.getPerimeter();
    }
    public float calculateArea() {
        return this.shape.getArea();
    }
}
```

- Rõ ràng từ ví dụ trên, ta thấy rõ : Nếu trường hợp cần mở rộng thêm các hình khác, ta chỉ cần tạo thêm class implement Shape mà không cần modify ShapeManager().

```
public static void main(String[] args) {
    Shape circle = new Circle(5);
    Shape square = new Square(5);
    Shape rectangle = new Rectangle(4, 6);

    ShapeManager manager = new ShapeManager();
    manager.setShape(circle);
    System.out.println("Circle with perimeter and area: "
        + manager.calculatePerimeter() + ":"
        + manager.calculateArea());

    manager.setShape(square);
    System.out.println("Square with perimeter and area: "
        + manager.calculatePerimeter() + ":"
        + manager.calculateArea());

    manager.setShape(rectangle);
    System.out.println("Rectangle with perimeter and area: "
        + manager.calculatePerimeter() + ":"
        + manager.calculateArea());
}
```

I Ví dụ IoC 2

```
public class A
{
    B b;
    public A()
    {
        b = new B();
    }
    public void Task1() {
        // do something here..
        b.SomeMethod();
        // do something here..
    }
}

public class B {
    public void SomeMethod() {
        //doing something..
    }
}
```

- Trong ví dụ trên, lớp A gọi `b.SomeMethod()` để hoàn thành nhiệm vụ `Task1` của mình. Lớp A không thể hoàn thành nhiệm vụ nếu không có lớp B và do đó có thể nói rằng "Lớp A phụ thuộc vào lớp B" hoặc "lớp B là sự phụ thuộc của lớp A".
- Trong phương pháp thiết kế OOP, các lớp cần tương tác với nhau để hoàn thành một hoặc nhiều chức năng của ứng dụng, chẳng hạn như trong ví dụ trên - lớp A và lớp B. Lớp A tạo và quản lý thời gian tồn tại của đối tượng của lớp B. Về cơ bản, nó kiểm soát việc tạo và thời gian tồn tại của các đối tượng của lớp phụ thuộc.

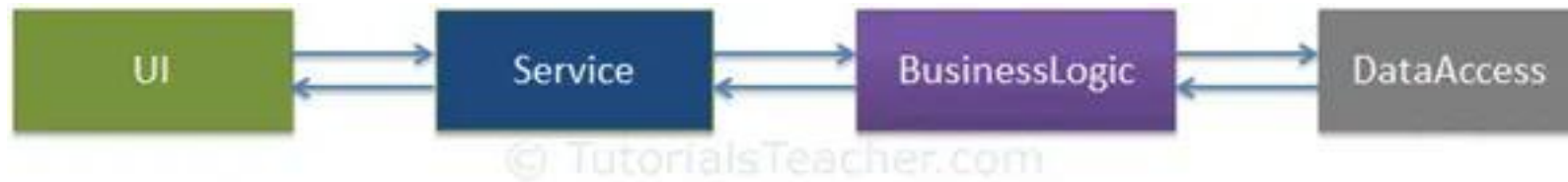
I Ví dụ IoC 2 (tt)

```
public class A
{
    B b;
    public A()
    {
        b = Factory.GetObjectOfB ();
    }
    public void Task1() {
        // do something here..
        b.SomeMethod();
        // do something here..
    }
}
public class Factory
{
    public static B GetObjectOfB()
    {
        return new B();
    }
}
```

- Nguyên tắc IoC gợi ý đảo ngược điều khiển. Điều này có nghĩa là chuyển giao điều khiển cho một lớp khác. Nói cách khác, đảo ngược điều khiển tạo phụ thuộc từ lớp A sang lớp khác.
- Như có thể thấy, lớp A sử dụng lớp Factory để lấy đối tượng của lớp B. Do đó, chúng ta đã **đảo ngược việc tạo đối tượng phụ thuộc từ lớp A sang Factory**.
- Lớp A không còn tạo đối tượng của lớp B nữa, thay vào đó, nó sử dụng lớp factory để lấy đối tượng của lớp B.

I Ví dụ IoC 3

- Ví dụ với mô hình N-tier điển hình:



```
//Truyền thống
public class CustomerBusinessLogic{
    DataAccess _dataAccess;
    public CustomerBusinessLogic(){
        _dataAccess = new DataAccess();
    }
    public string GetCustomerName(int id){
        return _dataAccess.GetCustomerName(id);
    }
}

public class DataAccess{
    public DataAccess(){}
    public string GetCustomerName(int id) {
        return "Dummy Customer Name";
        // get it from DB in real app
    }
}
```

- Có thể thấy, lớp **CustomerBusinessLogic** phụ thuộc vào lớp **DataAccess**. Nó tạo ra một đối tượng của lớp **DataAccess** để lấy dữ liệu KH. **Nhận xét:**
- **CustomerBusinessLogic** và **DataAccess** các lớp là các lớp được liên kết chặt chẽ. Vì vậy, những thay đổi trong **DataAccess** sẽ dẫn đến những thay đổi trong **CustomerBusinessLogic**. Ví dụ, nếu thêm, xóa hoặc đổi tên bất kỳ phương thức nào trong **DataAccess** thì cần thay đổi **CustomerBusinessLogic** cho phù hợp.
- Lớp **CustomerBusinessLogic** tạo ra một đối tượng của **DataAccess** bằng từ khóa **new**. Có thể có nhiều lớp sử dụng **DataAccess** và tạo ra các đối tượng của nó. Vì vậy, nếu thay đổi tên của lớp, thì cần tìm tất cả các vị trí trong mã nguồn đã tạo ra các đối tượng **DataAccess** thực hiện thay đổi trong toàn bộ mã.

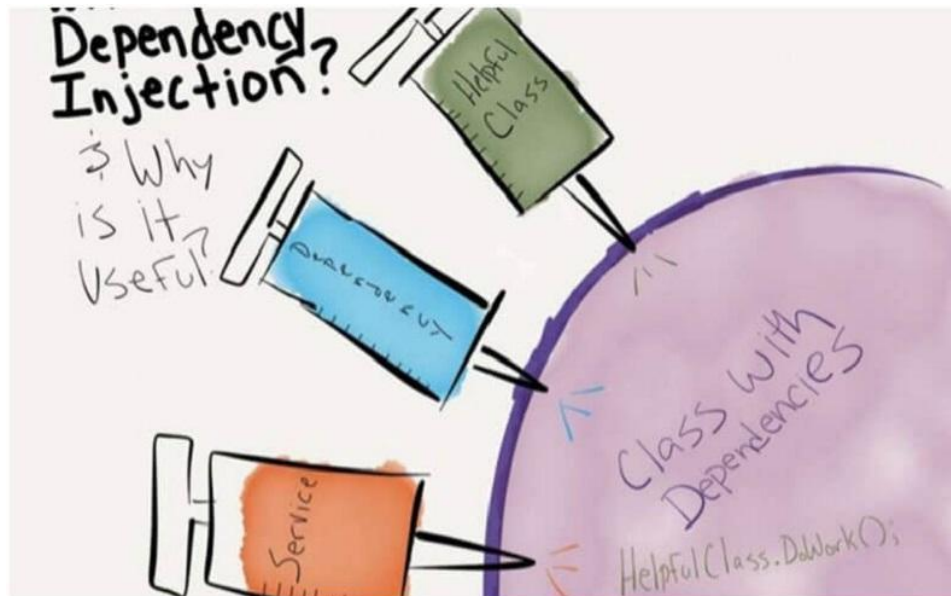
I Ví dụ IoC 3 (tt)

```
//Giải quyết bằng nguyên lý IoC
public class DataAccessFactory{
    public static DataAccess GetDataAccessObj() {
        return new DataAccess();
    }
}
public class CustomerBusinessLogic{
    public CustomerBusinessLogic(){
    }
    public string GetCustomerName(int id){
        DataAccess _dataAccess =
DataAccessFactory.GetDataAccessObj();
        return _dataAccess.GetCustomerName(id);
    }
}
```

- Như có thể thấy, lớp `CustomerBusinessLogic` sử dụng phương thức `GetDataAccessObj()` của lớp `DataAccessFactory` để lấy một đối tượng của lớp `DataAccess` thay vì tạo nó bằng từ khóa `new`.
- Do đó, chúng ta đã đảo ngược quyền kiểm soát việc tạo một đối tượng của một lớp phụ thuộc từ lớp `CustomerBusinessLogic` này sang `DataAccessFactory`.
- Từ đó đạt được sự liên kết lỏng lẻo giữa các lớp.

I 9.3 Dependency Injection (DI)

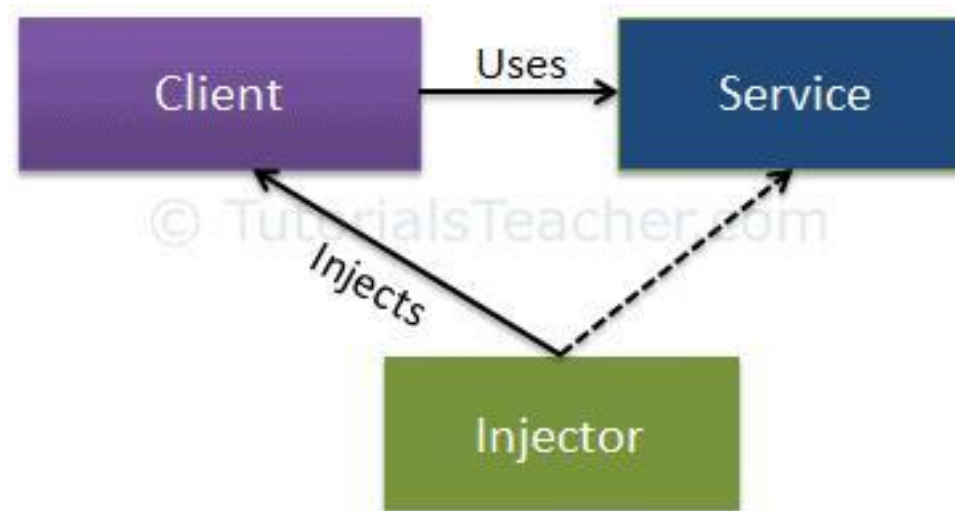
- **Dependency Injection (DI)** là một trong những pattern để implement **Dependency Inversion**, nó là một mô hình triển khai từ nguyên lý **IoC**, là một kỹ thuật trong lập trình trong đó một đối tượng cung cấp những phụ thuộc (dependency - là đối tượng, dịch vụ, chức năng) của đối tượng khác. Injection - Bơm vào (tiêm vào) ám chỉ một phụ thuộc (đối tượng, dịch vụ) đưa vào đối tượng để đối tượng sử dụng nó.
- Nguyên tắc cơ bản của DI là làm cho **high-level** module phụ thuộc vào **low-level** module thông qua injector, hay nói cách khác, *muốn tạo instance high-level module, ta phải tạo instance của low-level module và inject nó vào high-level module thông qua injector*. Injector ở đây có thể là constructor, setter hay interface.



I Dependency Injection (DI)

Mô hình **Dependency Injection** bao gồm 3 loại lớp.

- Lớp Client: Lớp khách hàng (lớp phụ thuộc) là lớp phụ thuộc vào lớp dịch vụ
- Lớp Service: Lớp dịch vụ (phụ thuộc) là lớp cung cấp dịch vụ cho lớp máy khách.
- Lớp Injector: Lớp injector sẽ đưa đối tượng lớp dịch vụ vào lớp máy khách.



I Các loại Dependency Injection

- **Constructor Injection**: Trình tiêm cung cấp dịch vụ (phụ thuộc) thông qua hàm khởi tạo lớp máy khách.
- **Property Injection**: Trong quá trình tiêm thuộc tính (hay còn gọi là **Setter Injection**), trình tiêm cung cấp sự phụ thuộc thông qua một thuộc tính công khai của lớp máy khách.
- **Method Injection**: Lớp máy khách triển khai một giao diện khai báo phương thức để cung cấp sự phụ thuộc và trình tiêm sử dụng giao diện này để cung cấp sự phụ thuộc cho lớp máy khách.

I Ví dụ DI - Constructor Injection

```
public interface ICustomerDataAccess{
    string GetCustomerName(int id);
}
public class CustomerDataAccess: ICustomerDataAccess{
    public CustomerDataAccess(){}
    public string GetCustomerName(int id)
    {
        //get the customer name from the db in real app
        return "Dummy Customer Name";
    }
}
public class CustomerBusinessLogic{
    ICustomerDataAccess _dataAccess;

    public CustomerBusinessLogic(ICustomerDataAccess custDataAccess){
        _dataAccess = custDataAccess;
    }

    public CustomerBusinessLogic() {
        _dataAccess = new CustomerDataAccess();
    }

    public string ProcessCustomerData(int id) {
        return _dataAccess.GetCustomerName(id);
    }
}
```

```
public class CustomerService{
    CustomerBusinessLogic _cust;

    public CustomerService(){
        _cust = new CustomerBusinessLogic(new CustomerDataAccess());
    }
    public string GetCustomerName(int id) {
        return _cust.ProcessCustomerData(id);
    }
}
```

- Có thể thấy, **CustomerService** tạo và inject đối tượng **CustomerDataAccess** vào lớp **CustomerBusinessLogic**.
- Lớp gọi (**CustomerService**) tạo và đặt lớp **CustomerDataAccess** thích hợp vào lớp **CustomerBusinessLogic**.
- Theo cách này, các lớp **CustomerBusinessLogic** và **CustomerDataAccess** trở thành các lớp "lỏng lẻo" hơn.

I Ví dụ DI - Property Injection

```
public class CustomerBusinessLogic{
    public CustomerBusinessLogic(){
    }

    public string GetCustomerName(int id){
        return DataAccess.GetCustomerName(id);
    }
    public ICustomerDataAccess DataAccess {
        get; set;
    }
}

public class CustomerService{
    CustomerBusinessLogic _customerBL;

    public CustomerService(){
        _customerBL = new CustomerBusinessLogic();
        _customerBL.DataAccess = new CustomerDataAccess();
    }

    public string GetCustomerName(int id) {
        return _customerBL.GetCustomerName(id);
    }
}
```

- Có thể thấy, lớp **CustomerBusinessLogic** bao gồm thuộc tính công khai có tên là **DataAccess**, nơi có thể thiết lập một thể hiện của lớp thực hiện **ICustomerDataAccess**. Vì vậy, **CustomerService** có thể tạo và thiết lập lớp **CustomerDataAccess** bằng thuộc tính công khai này.

I Ví dụ DI - Method Injection

```
interface IDataAccessDependency{
    void SetDependency(ICustomerDataAccess customerDataAccess);
}

public class CustomerBusinessLogic : IDataAccessDependency{
    ICustomerDataAccess _dataAccess;
    public CustomerBusinessLogic(){}

    public string GetCustomerName(int id){
        return _dataAccess.GetCustomerName(id);
    }

    public void SetDependency(ICustomerDataAccess
customerDataAccess){
        _dataAccess = customerDataAccess;
    }
}
```

```
public class CustomerService{
    CustomerBusinessLogic _customerBL;
    public CustomerService(){
        _customerBL = new CustomerBusinessLogic();
        ((IDataAccessDependency)_customerBL).SetDependency(new
CustomerDataAccess());
    }
    public string GetCustomerName(int id){
        return _customerBL.GetCustomerName(id);
    }
}
```

- Trong ví dụ trên, lớp **CustomerBusinessLogic** triển khai giao diện **IDataAccessDependency**, bao gồm phương thức **SetDependency()**.
- Vì vậy, lớp injector **CustomerService** sẽ sử dụng phương thức này để inject lớp phụ thuộc (**CustomerDataAccess**) vào lớp client.

I Ưu nhược điểm của Dependency Injection

Ưu điểm	Nhược điểm
- Giảm sự kết dính giữa các module. - Code dễ bảo trì, dễ thay thế module - Rất dễ test và viết Unit Test. - Dễ dàng thấy quan hệ giữa các module (Vì các dependency đều được inject vào constructor).	- Khái niệm DI khá “khó tiêu”, các developer mới sẽ gặp khó khăn khi học. - Sử dụng interface nên đôi khi sẽ khó debug, do không biết chính xác module nào được gọi. - Các object được khởi tạo toàn bộ ngay từ đầu, có thể làm giảm performance - Làm tăng độ phức tạp của code.

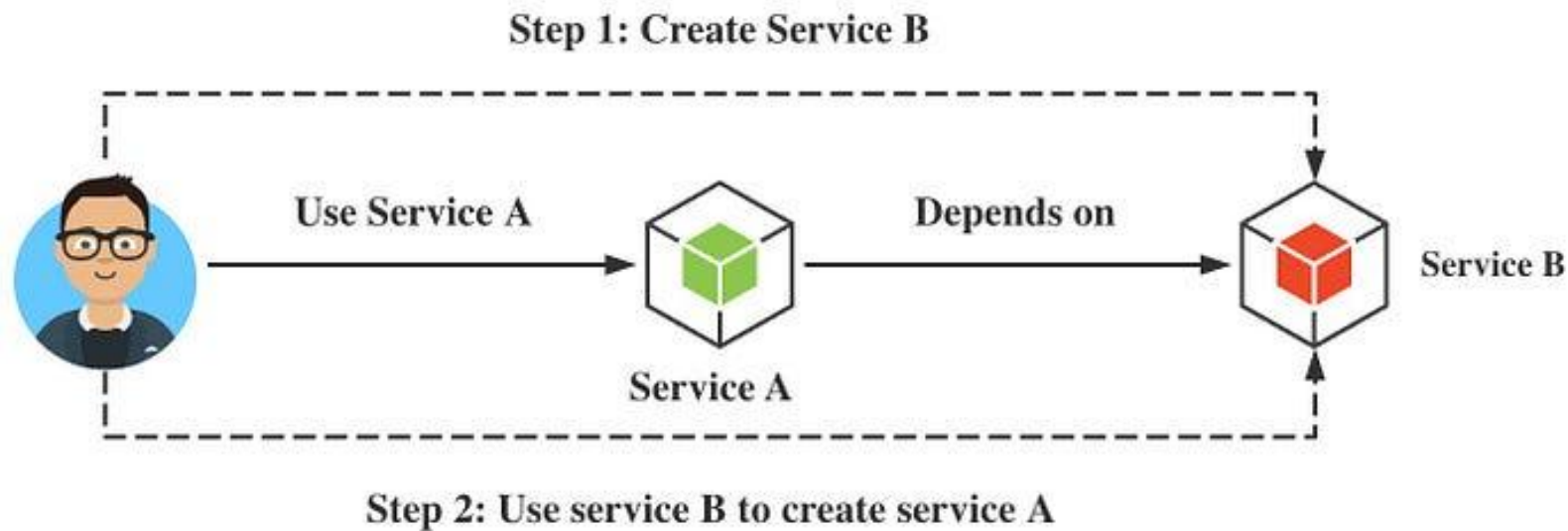
| 9.4 IoC Container

- **IoC Container** (hay còn gọi là DI Container) là một khuôn khổ để triển khai dependency injection tự động. Nó quản lý việc tạo đối tượng và thời gian tồn tại của đối tượng, đồng thời inject các dependency vào lớp.
- **IoC Container** tạo ra một đối tượng của lớp được chỉ định và cũng inject tất cả các đối tượng phụ thuộc thông qua một constructor, một thuộc tính hoặc một phương thức tại thời điểm chạy và loại bỏ nó tại thời điểm thích hợp. Điều này được thực hiện để chúng ta không phải tạo và quản lý các đối tượng theo cách thủ công.

| IoC Container

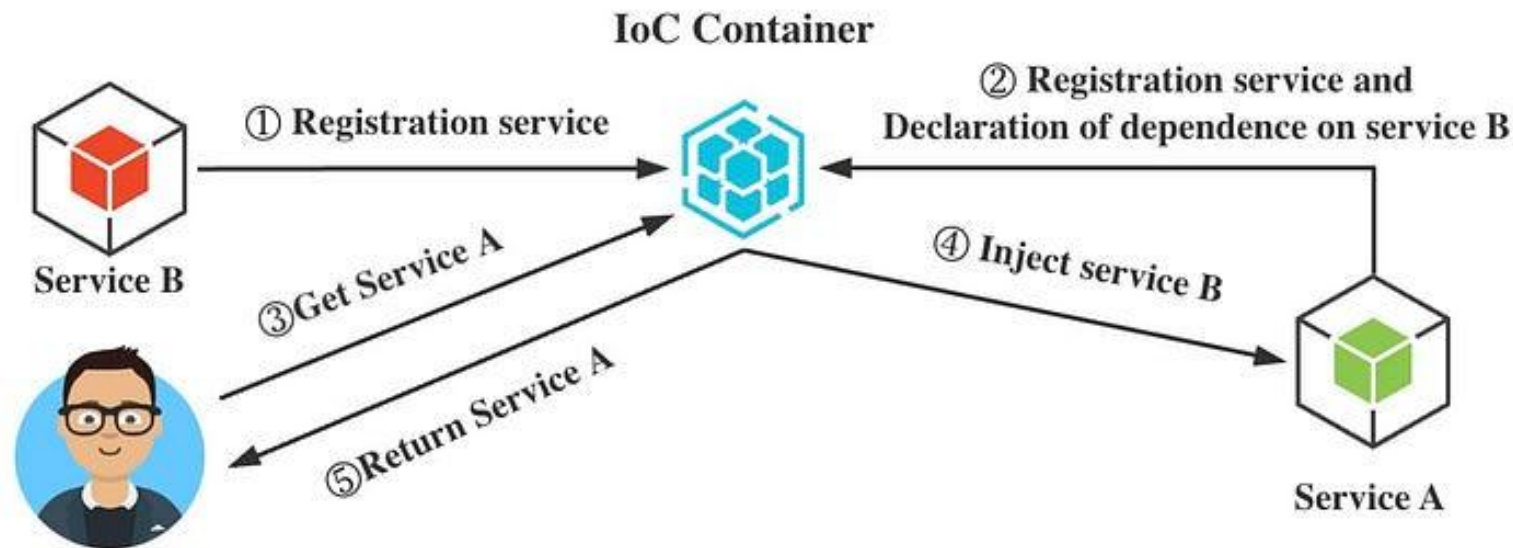
Không sử dụng IoC Container:

- Giả sử ta có Service A phụ thuộc Service B.
- Chúng ta cần tạo Service B trước khi tạo Service A, sơ đồ các bước như sau:



I Sử dụng IoC Container

- Các dịch vụ trong hệ thống sẽ được đăng ký vào **IoC container** một cách thống nhất, và nếu dịch vụ có sự phụ thuộc vào các dịch vụ khác thì cần được khai báo với IoC Container.
- Khi người dùng cần sử dụng một dịch vụ cụ thể, IoC Container sẽ chịu trách nhiệm tạo và quản lý dịch vụ và các đối tượng phụ thuộc của nó. Quy trình cụ thể được thể hiện trong sơ đồ sau.

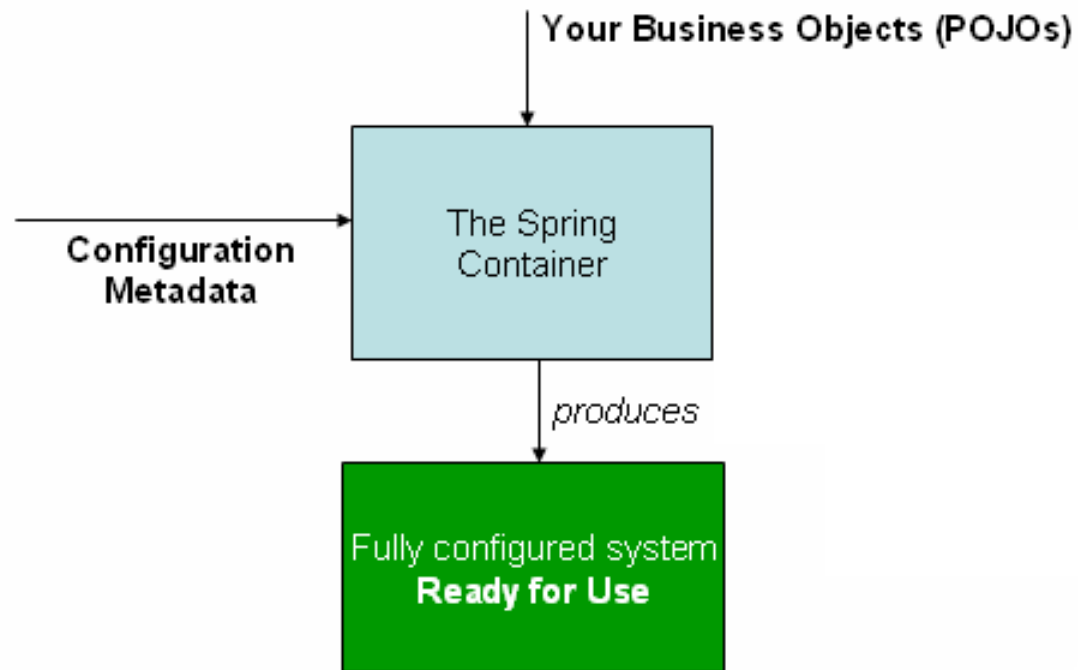


| IoC Container trong .NET

- Unity
- StructureMap
- Castle Windsor
- Ninject
- Autofac
- DryIoc
- Simple Injector
- Light Inject

| IoC Container trong Java

- Spring IoC Container



I Ví dụ IoC Container với Unity trong .NET

```
public interface ICar
{
    int Run();
}
public class BMW : ICar
{
    private int _miles = 0;
    public int Run()
    {
        return ++_miles;
    }
}
public class Ford : ICar
{
    private int _miles = 0;
    public int Run()
    {
        return ++_miles;
    }
}
```

```
public class Audi : ICar
{
    private int _miles = 0;
    public int Run()
    {
        return ++_miles;
    }
}
public class Driver
{
    private ICar _car = null;
    public Driver(ICar car)
    {
        _car = car;
    }

    public void RunCar()
    {
        Console.WriteLine("Running {0} - {1} mile ",
            _car.GetType().Name, _car.Run());
    }
}
```

I Ví dụ IoC Container với Unity trong .NET (tt)

```
IUnityContainer container = new UnityContainer();  
// Map ICar with BMW  
container.RegisterType<ICar, BMW>();  
  
//Resolves dependencies and returns the Driver object  
Driver drv = container.Resolve<Driver>();  
drv.RunCar();  
//Output: Running BMW - 1 mile
```

- Trong ví dụ trên, vùng `UnityContainer` tạo một đối tượng của lớp `Driver` bằng phương thức `container.Resolve<Driver>()`.
- Lớp `Driver` là một phụ thuộc của `ICar`. Vì vậy, `container.Resolve<Driver>()` trả về một đối tượng của `Driver` lớp bằng cách tự động tạo và chèn một đối tượng `BMW` vào đó.
- Tất cả những điều này đều trong suốt với người dùng và do IoC Container thực hiện. Đối tượng `BMW` được tạo và chèn vì kiểu của `BMW` là `ICar`.

I Ví dụ IoC Container với Unity trong .NET (tt)

//Container Unity sẽ tạo một đối tượng mới và inject nó mỗi khi resolve cùng một kiểu.

```
var container = new UnityContainer();  
container.RegisterType<ICar, BMW>();
```

```
Driver driver1 = container.Resolve<Driver>();  
driver1.RunCar();
```

```
Driver driver2 = container.Resolve<Driver>();  
driver2.RunCar();
```

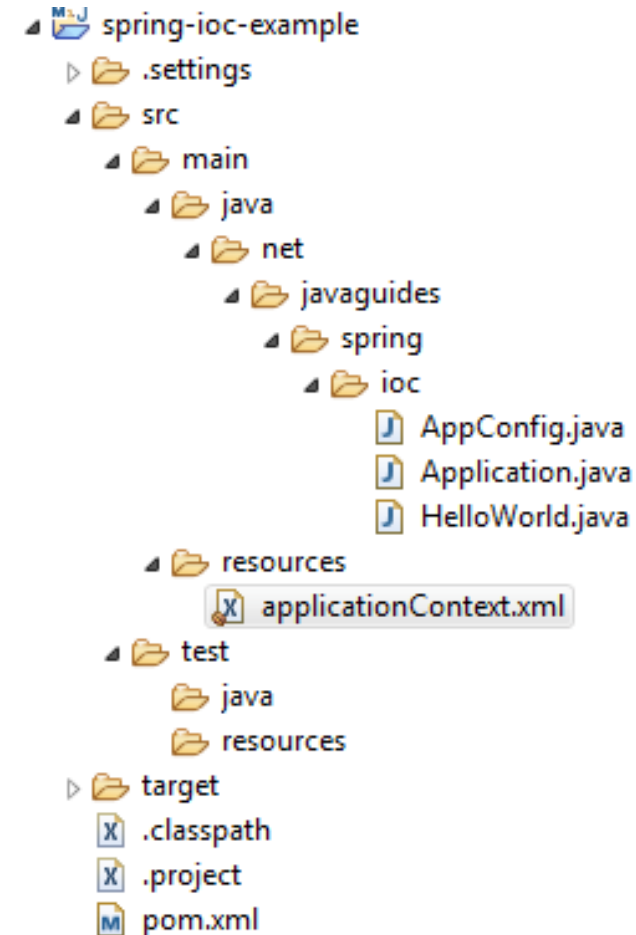
//Output:

//Running BMW - 1 mile

//Running BMW - 1 mile

I Ví dụ IoC Container với Spring Java

- Tạo 1 dự án Maven Project
- Thêm Maven Dependencies
- Cấu hình HelloWorld Spring Bean
- Tạo Spring Container
- Nhận Beans từ Spring Container



I Ví dụ IoC Container với Spring Java (tt)

- Add Maven Dependencies vào file pom.xml:

```
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>6.1.8</version>
</dependency>
```

- Tạo lớp HelloWorld:

```
public class HelloWorld {
    private String message;
    public void setMessage(String message) {
        this.message = message;
    }
    public void getMessage() {
        System.out.println("My Message: " + message);
    }
}
```

I Ví dụ IoC Container với Spring Java (tt)

- Cấu hình lớp `HelloWorld` này thành một Spring bean:

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class AppConfig {
    @Bean
    public HelloWorld helloWorld() {
        HelloWorld helloWorld = new HelloWorld();
        helloWorld.setMessage("Hello World!");
        return helloWorld;
    }
}
```

I Ví dụ IoC Container với Spring Java (tt)

- Tạo một Spring Container: Tạo một đối tượng container Spring bằng cách sử dụng lớp `AnnotationConfigApplicationContext` triển khai từ giao diện `ApplicationContext`.

```
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Application {
    public static void main(String[] args) {
        AnnotationConfigApplicationContext context =
            new AnnotationConfigApplicationContext(AppConfig.class);
        context.close();
    }
}
```

I Ví dụ IoC Container với Spring Java (tt)

- Lấy Bean từ Spring Container: Giao diện `ApplicationContext` cung cấp phương thức `getBean()` để lấy bean từ vùng chứa Spring.

```
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
public class Application {
    public static void main(String[] args) {
        AnnotationConfigApplicationContext context =
            new AnnotationConfigApplicationContext(AppConfig.class);
        HelloWorld obj = (HelloWorld) context.getBean("helloWorld");
        obj.getMessage();
        context.close();
    }
}
```

- Kết quả:

My Message: Hello World!

| Sinh viên tham khảo thêm

- <https://www.tutorialsteacher.com/ioc/introduction>
- <https://www.baeldung.com/inversion-control-and-dependency-injection-in-spring>
- <https://docs.spring.io/spring-framework/reference/core/beans/dependencies/factory-collaborators.html>
- <https://docs.spring.io/spring-framework/reference/core/beans.html>